

คู่มือปฏิบัติการ AG-101 — Antigravity 2.0 สำหรับพนักงานสำนักงาน

ตอนที่ 4: สร้างแอปพลิเคชันจริงด้วย Google Apps Script

ส่งเอเจนต์เขียนระบบลงทะเบียนอบรมทั้งหน้าเว็บและหลังบ้าน แล้วนำขึ้นใช้งานจริงบน Google Sheets

เวลาโดยประมาณ: 75 นาที | ระดับ: ไม่ต้องมีพื้นฐานเขียนโปรแกรม

1 ภาพรวมของเรื่องนี้

ฝ่ายบุคคลขอให้คุณทำ “ระบบลงทะเบียนอบรมออนไลน์” ที่บันทึกข้อมูลลง Google Sheets พร้อมแดชบอร์ดสถิติ — โดยคุณ **ไม่ต้องเขียนโค้ดเองแม้แต่บรรทัดเดียว** คุณเป็นผู้กำหนดความต้องการ (Requirement) ส่วนเอเจนต์ (Agent) ของ Antigravity 2.0 เป็นวิศวกรผู้สร้างให้ทั้งหมด

เรื่องนี้แบ่งเป็น 3 ส่วน

1. **สร้าง** — ส่งเอเจนต์เขียนไฟล์ 2 ไฟล์: สคริปต์หลังบ้าน `api-handler.gs` และหน้าเว็บ `registration-portal.html`
2. **นำขึ้นใช้งานจริง (Deploy)** — เลือกแทริก A (คัดลอก-วาง เหมาะกับทุกคน) แทริก B (ใช้เครื่องมือ CLASP ให้เอเจนต์ส่งโค้ดขึ้นคลาวด์เอง) หรือแทริก C (ทุกอย่างอยู่บน Google ทั้งหมด)
3. **ทดสอบปิดงาน** — กรอกฟอร์มจริง แล้วเห็นข้อมูลวิ่งเข้า Google Sheets

2 สิ่งที่ต้องเตรียม

- Antigravity 2.0 Desktop App ที่ล็อกอินแล้ว และการเชื่อมต่ออินเทอร์เน็ต
- บัญชี Google (Gmail) ที่เข้าใช้ Google Sheets ได้
- เฉพาะแทริก B: ติดตั้ง Node.js ไว้ในเครื่องแล้ว

3 ไฟล์ในโฟลเดอร์

ไฟล์	คำอธิบาย
<code>sample-registrations.csv</code>	โครงสร้างข้อมูลผู้ลงทะเบียนตัวอย่าง 5 คอลัมน์
<code>deploy-guide.md</code>	คู่มือนำขึ้นใช้งานฉบับเต็ม (อ้างอิงเพิ่มเติมจากคู่มืออื่น)

4 ส่วนที่ 1: สั่งเอเจนต์สร้างระบบ

ขั้นตอนที่ 1 เปิดโฟลเดอร์เคสเป็นพื้นที่ทำงาน

เปิด Antigravity 2.0 เลือกเปิดโฟลเดอร์ (Open Folder) ซึ่ไปที่ case-4-appscript-app ทั้งโฟลเดอร์ แล้วเปิดดู sample-registrations.csv ให้เห็นว่าฟอร์มต้องเก็บข้อมูล 5 ช่อง: ชื่อ-สกุล, อีเมล, เบอร์โทร, อาชีพ และหัวข้อที่สนใจ

ขั้นตอนที่ 2 สัการกิจที่ 1: สร้างระบบทั้งหมดในพรอมต์เดียว

วางพรอมต์ (Prompt) ต่อไปนี้แล้วกดสั่งรัน (Run)

พรอมต์ — พิมพ์หรือคัดลอกไปวางในช่องสั่งงานของ Antigravity — การกิจที่ 1

จากโครงสร้างข้อมูลในไฟล์ sample-registrations.csv ให้สร้างระบบลงทะเบียนอบรมและแดชบอร์ดดังนี้:

1. [api-handler.gs] ฟังก์ชัน doPost(e) รับข้อมูลลงทะเบียนแบบ JSON (ชื่อ-สกุล, อีเมล, เบอร์โทร, อาชีพ, หัวข้อที่สนใจ) บันทึกลงแถวถัดไปของ Google Sheets แบน Timestamp คอลัมน์แรก ตอบกลับ JSON {status:"success"} และรองรับ CORS
2. [registration-portal.html] หน้าเว็บแบ่งสองฝั่ง: ฝั่งซ้ายฟอร์มลงทะเบียน ฝั่งขวาแดชบอร์ด Chart.js (กราฟแท่งตามอาชีพ + กราฟวงกลมตามหัวข้อที่สนใจ) มีช่องวาง Apps Script URL, ปุ่มส่งข้อมูลแบบ Fetch API ไม่รีเฟรชหน้า, Loading Spinner, Toast แจ้งผล ดีไซน์ Minimalist Glassmorphism โทนพาสเทลม่วง-ฟ้า ฟอนต์ Outfit + Sarabun
3. บันทึกทั้งสองไฟล์ลงในโฟลเดอร์นี้

ผลลัพธ์ที่ควรได้ในขั้นตอนนี้

ได้ไฟล์ใหม่ 2 ไฟล์ในโฟลเดอร์: api-handler.gs (สคริปต์หลังบ้าน ที่จะไปรันบน Google) และ registration-portal.html (หน้าเว็บสำหรับผู้ใช้) ลองดับเบิลคลิกเปิดหน้าเว็บดูได้เลย — ฟอร์มจะแสดงผลแล้ว แต่ปุ่มส่งข้อมูลยังทำงานไม่ได้จนกว่าจะผ่านส่วนที่ 2

5 ส่วนที่ 2 แทร็ก A: นำขึ้นใช้งานแบบคัดลอก-วาง (ทุกคนทำ)

ขั้นตอนที่ 3 สร้าง Google Sheet และเปิด Apps Script

1. เข้าเบราว์เซอร์ ไปที่ Google Drive สร้าง **Google Sheets** แผ่นใหม่ ตั้งชื่อ เช่น “ลงทะเบียนอบรม AG-101”
2. ในแผ่นชีต คลิกเมนู **ส่วนขยาย (Extensions)** เลือก **Apps Script** จะเปิดหน้าเขียนสคริปต์ขึ้นมาใหม่

3. กลับมาที่ Antigravity เปิดไฟล์ `api-handler.gs` คัดลอกโค้ด ทั้งหมด ไปวางทับโค้ดเดิมในหน้า Apps Script
4. กดปุ่มบันทึก (ไอคอนแผ่นดิสก์)

ขั้นตอนที่ 4 เปิดใช้งานเป็นเว็บแอป (Deploy)

1. คลิกปุ่ม การทำให้ใช้งานได้ (Deploy) มุมขวาบน เลือก การทำให้ใช้งานได้ใหม่ (New deployment)
2. คลิกไอคอนฟันเฟือง เลือกประเภท เว็บแอป (Web app)
3. ตั้งค่า: เรียกใช้ในฐานะ = **ฉัน (Me)** และผู้มีสิทธิ์เข้าถึง = **ทุกคน (Anyone)** แล้วกด Deploy
4. ระบบจะขอสิทธิ์: กด **Authorize access** เลือกบัญชี Google ของคุณ หากขึ้นจอเตือน ให้กด **Advanced** แล้วกด **Go to ... (unsafe)** และ **Allow**
5. คัดลอก **Web App URL** ที่ได้ เก็บไว้ใช้ขั้นถัดไป

ข้อควรระวัง

ต้องตั้งผู้มีสิทธิ์เข้าถึงเป็น “ทุกคน (Anyone)” เท่านั้น มิฉะนั้น หน้าเว็บของเราจะส่งข้อมูลข้ามมายังสคริปต์ไม่ได้ (ติดบล็อกสิทธิ์) — คำเตือน unsafe เป็นเรื่องปกติของสคริปต์ที่เราเขียนเอง ซึ่งยังไม่ผ่านการรับรองจาก Google

ขั้นตอนที่ 5 เชื่อมหน้าเว็บเข้ากับชิตและทดสอบ

1. เปิด `registration-portal.html` ในเบราว์เซอร์
2. วาง Web App URL ที่คัดลอกไว้ลงช่อง “Apps Script URL” บนหน้าเว็บ
3. กรอกข้อมูลทดสอบ 1 รายการ แล้วกดปุ่มลงทะเบียน จะเห็นสถานะกำลังส่งข้อมูล ตามด้วยแจ้งผลสำเร็จ
4. กลับไปดูหน้า Google Sheets

ผลลัพธ์ที่ควรได้ในขั้นตอนนี้

ผลลัพธ์สุดท้ายของเทร็ก A — แถวใหม่ปรากฏใน Google Sheets พร้อมวันเวลาประทับ (Timestamp) ในคอลัมน์แรก และข้อมูลที่กรอกครบ 5 ช่อง ลองให้เพื่อนข้าง ๆ เปิดหน้าเว็บของคุณแล้วลงทะเบียนเพิ่มอีกคนก็ได้ — นี่คือแอปพลิเคชันจริงที่ใช้งานได้บนอินเทอร์เน็ต

6 ส่วนที่ 2 แทร็ก B: ให้เอเจนต์ส่งโค้ดขึ้นคลาวด์เองด้วย CLASP (ขั้นสูง)

แทร็กนี้แสดงการปิดวงจรอัตโนมัติเต็มรูปแบบ: เอเจนต์แก้โค้ดในเครื่องแล้ว “ผลัก (push)” ขึ้น Google Apps Script เองผ่านเครื่องมือ CLASP ของ Google โดยเราไม่ต้องคัดลอก-วางอีกต่อไป

ขั้นตอนที่ 6 ติดตั้งและเชื่อมบัญชี CLASP (ทำครั้งแรกครั้งเดียว)

เปิด Terminal (หรือ Command Prompt) แล้วรันทีละคำสั่ง

```
npm install -g @google/clasp
clasp login
```

คำสั่งที่สองจะเปิดเบราว์เซอร์ให้เลือกบัญชี Google และกดอนุญาตสิทธิ์

ข้อควรระวัง

ก่อนใช้ CLASP ต้องเปิดสวิตช์ **Google Apps Script API** ก่อน โดยเข้า script.google.com/home/usersettings แล้วเปิด (On) — ถ้าไม่เปิด คำสั่ง clasp จะฟ้องข้อผิดพลาด 403

ขั้นตอนที่ 7 ผูกโฟลเดอร์กับโปรเจกต์สคริปต์

ไปที่หน้า Apps Script ของซีต (จากแทร็ก A) คลิกไอคอนฟันเฟือง **Project Settings** คัดลอก **Script ID** แล้วรันในโฟลเดอร์ case-4-appscript-app (แทนคำว่า SCRIPT_ID ด้วยรหัสที่คัดลอกมา)

```
clasp clone "SCRIPT_ID"
```

ขั้นตอนที่ 8 ส่งเอเจนต์ปิดงานเองทั้งวงจร

กลับมาที่ Antigravity แล้ววางพรมต์

พรมต์ — พิมพ์หรือคัดลอกไปวางในช่องสั่งงานของ Antigravity — แทร็ก B

เขียนโค้ดจาก api-handler.gs ลงไฟล์สคริปต์ของโปรเจกต์ CLASP ในโฟลเดอร์นี้ จากนั้นรันคำสั่ง clasp push เพื่ออัปเดตโค้ดขึ้น Google Apps Script ให้เลย โดยไม่ต้องให้ฉันทัดลอกวางเอง เสร็จแล้วสรุปว่าต้องกด Deploy ใหม่ที่หน้าไหน

ผลลัพธ์ที่ควรได้ในขั้นตอนนี้

เอเจนต์เขียนไฟล์และรัน clasp push เอง — เปิดหน้า Apps Script ในเบราว์เซอร์จะเห็นโค้ดเปลี่ยนตามทันที จากนั้นกด **Deploy** เลือก **Manage deployments** กดไอคอนดินสอ เลือก **New version** แล้ว Deploy ซ้ำหนึ่งครั้ง เพื่อให้เวอร์ชันใหม่ขึ้นใช้งานจริง

7 ส่วนที่ 2 แทร็ก C: ทุกอย่างอยู่บน Google ทั้งหมด (ต่อยอดจาก แทร็ก B)

ในแทร็ก A และ B หน้าเว็บยังเป็นไฟล์ที่เปิดจากเครื่องของเรา แทร็ก C ย้าย *ทุกส่วน* ขึ้นคลาวด์: หน้าเว็บถูกโฮสต์ในโปรเจกต์ Apps Script เอง (เสิร์ฟสดจาก script.google.com) และคุยกับซีตผ่านกลไกเรียกภายในของ Google (google.script.run) จึงตัดปัญหาการบล็อกสิทธิ์ข้ามเว็บ (CORS) ทั้งทั้งหมด — แนวทางนี้ผ่านการทดสอบจริงกับ Antigravity แล้ว

ขั้นตอนที่ 9 สร้างเอเจนต์สร้างและย้ายระบบขึ้น Google ทั้งหมด

เงื่อนไข: ผ่าน clasp login และเปิด Google Apps Script API แล้ว (ขั้นตอนที่ 6) จากนั้นวางพร้อมดังนี้

พร้อม — พิมพ์หรือคัดลอกไปวางในช่องสั่งงานของ Antigravity — แทร็ก C

สร้างโปรเจกต์ Google Sheets ใหม่บนบัญชีของฉันด้วยคำสั่ง clasp create --type sheets --title "แอปพลิเคชันลงทะเบียนอบรม" แล้วดำเนินการดังนี้:

1. ย้ายหน้าเว็บ registration-portal.html เข้าไปเป็นไฟล์ HTML ในโปรเจกต์ Apps Script และเพิ่มฟังก์ชัน doGet() ที่เสิร์ฟหน้านี้ด้วย HtmlService
2. เปลี่ยนการส่งข้อมูลจาก Fetch API เป็น google.script.run เพื่อให้หน้าเว็บคุยกับซีตผ่านระบบภายในของ Google โดยไม่ติดปัญหา CORS
3. รัน clasp push แล้ว deploy เป็น Web App (สิทธิ์ Anyone)
4. รายงานลิงก์ทั้ง 3 รายการ: Web App URL, Google Sheets และหน้า Apps Script Editor

ผลลัพธ์ที่ควรได้ในขั้นตอนนี้

เอเจนต์จะสร้างซีต เขียนสคริปต์ และ deploy เองครบวงจร แล้วรายงานลิงก์กลับมา 3 รายการ: (1) **Web App URL** — หน้าเว็บลงทะเบียนและแดชบอร์ด เสิร์ฟจาก Google เปิดจากเครื่องใดก็ได้ (2) **Google Sheets** — ตารางเก็บข้อมูลจริง (3) **Apps Script Editor** — หน้าโค้ดทั้งหมดบนคลาวด์ ทดสอบปิดงาน: เปิด Web App URL กรอกฟอร์ม แล้วเห็นข้อมูลเข้าซีตพร้อม Timestamp โดยไม่เหลือไฟล์ที่ต้องเปิดจากเครื่องเลย

เคล็ดลับ

ระหว่างทางเอเจนต์อาจหยุดถาม 2 จังหวะ: (1) เบราวเซอร์เปิดหน้า OAuth ให้เลือกบัญชีและอนุญาตสิทธิ์ (2) หากยังไม่เปิด Google Apps Script API เอเจนต์จะแจ้งลิงก์ให้ไปเปิด — เปิดแล้วตอบกลับว่า “เรียบร้อยแล้ว” เอเจนต์จะทำงานต่อโดยอัตโนมัติ

8 เกณฑ์ผ่านเคสนี้

1. เอเจนต์สร้าง `api-handler.gs` และ `registration-portal.html` สำเร็จ
2. ลงทะเบียนผ่านหน้าเว็บแล้วข้อมูลเข้า Google Sheets จริงพร้อม Timestamp
3. (เฉพาะผู้ทำแทริก B) เอเจนต์ push โค้ดขึ้นคลาวด์ได้เองสำเร็จ
4. (เฉพาะผู้ทำแทริก C) เปิดหน้าเว็บจาก URL ของ `script.google.com` ได้โดยตรง — ระบบทั้งหมดอยู่บน Google โดยไม่เหลือไฟล์ฝั่งเครื่อง

9 การกิจเสริม (สำหรับผู้เสร็จเร็ว)

- ส่งเอเจนต์เพิ่มการตรวจอีเมลซ้ำ (ห้ามลงทะเบียนอีเมลเดิมสองครั้ง) ทั้งฝั่งหน้าเว็บและฝั่งสคริปต์ แล้ว Deploy เวอร์ชันใหม่
- ส่งเอเจนต์เพิ่มปุ่มส่งออก (Export) รายชื่อผู้ลงทะเบียนเป็นไฟล์ CSV จากแดชบอร์ด

10 ปัญหาที่พบบ่อย

อาการ	สาเหตุที่พบบ่อย	วิธีแก้
กดส่งแล้วขึ้นข้อผิดพลาดเรื่อง CORS หรือส่งไม่ออก	สิทธิ์เข้าถึงเว็บแอปไม่ใช่ Anyone หรือใช้ URL เวอร์ชันเก่า	Deploy ใหม่ ตรวจสอบว่าเลือก Anyone แล้ว คัดลอก URL ล่าสุดมาวาง
ข้อมูลไม่เข้าชีต แต่หน้าเว็บแจ้งสำเร็จ	วางโค้ดลงคนละโปรเจกต์กับชีต	สร้าง Apps Script จากเมนูในชีตโดยตรง (Extensions) เท่านั้น
clasp พ้อง 403 หรือ Permission denied	ยังไม่เปิด Google Apps Script API	เปิดที่ script.google.com/home/usersettings แล้วรอ 1–2 นาทีค่อยลองใหม่
พิมพ์ npm แล้วบอกไม่รู้จักคำสั่ง	ยังไม่ได้ติดตั้ง Node.js	ติดตั้ง Node.js LTS จาก nodejs.org แล้วเปิด Terminal ใหม่
push แล้ว หน้าเว็บ ยัง ได้ พฤติกรรมเดิม	ยังไม่กด Deploy เวอร์ชันใหม่	Manage deployments เลือก New version ตามท้ายแท็บ B

ข้อควรระวัง

บทเรียนปิดคอร์ส: เคสนี้คือภาพรวมของการทำงานร่วมกับ AI ยุคใหม่ — มนุษย์กำหนดโจทย์ ตรวจสอบรับงาน และรับผิดชอบการเผยแพร่ ส่วนเอเจนต์เขียนได้ทั้งหลังบ้าน หน้าบ้าน และส่งงานขึ้นคลาวด์เอง ระบบที่เปิดสิทธิ์ “Anyone” คือระบบสาธารณะจริง อย่าใช้เก็บข้อมูลอ่อนไหวโดยไม่ปรึกษาฝ่ายไอทีขององค์กร